セマンティック検索API仕様書

作成日: 2026-03-06 15:38
最終更新: 2026-03-06 15:38
バージョン: 1.0.0
対象画面: プロジェクト管理画面 - トランジション検索モーダル
総API数: 9個
実装状況: 9個実装済み、0個未実装

📊 実装状況サマリー

カテゴリ	実装済み	未実装	合計
検索	2	0	2
インデックス管理	2	0	2
バッチ処理	4	0	4
ヘルスチェック	1	0	1
合計	**9**	**0**	**9**

完了率: 100% (9/9)

🎯 API一覧

#	API名	メソッド	エンドポイント	状態
1	セマンティック検索	POST	`/api/v1/semantic-search/search`	✅ 実装済み
2	クエリ分解（AI4軸分解）	POST	`/api/v1/semantic-search/decompose`	✅ 実装済み
3	ファイルインデックス登録	POST	`/api/v1/semantic-search/index`	✅ 実装済み
4	インデックス統計取得	GET	`/api/v1/semantic-search/stats`	✅ 実装済み
5	ヘルスチェック	GET	`/api/v1/semantic-search/health`	✅ 実装済み
6	バッチ処理開始	POST	`/api/v1/semantic-search/batch`	✅ 実装済み
7	バッチジョブ一覧取得	GET	`/api/v1/semantic-search/batch`	✅ 実装済み

8	バッチ統計取得	GET	`/api/v1/semantic-search/batch/stats`	✅ 実装済み
9	バッチジョブキャンセル	DELETE	`/api/v1/semantic-search/batch/:jobId`	✅ 実装済み

📋 概要

セマンティック検索APIは、MAIMLファイル内のトランジションセットを4軸ベクトル（入力条件・手順・出力・構造）でEmbedding化し、自然言語クエリによる類似度検索を提供します。

主な特徴

1. **4軸ベクトル検索**: 入力条件(input)・手順(transition)・出力(output)・構造(structure)の4次元でトランジションを評価
2. **AI自然言語分解**: GPTによるクエリ自動分解（4軸への振り分け）
3. **重み調整**: 4軸ごとの検索重みをカスタマイズ可能
4. **バッチ処理**: 組織内全MAIMLファイルの一括Embedding生成

使用コンポーネント

コンポーネント	ファイル	使用API
トランジション検索モーダル	`TransitionSetSearchModal.tsx`	#1, #2
バッチ処理パネル	`TransitionSetBatchProcessingPanel.tsx`	#6, #7, #8, #9
セマンティック検索サービス	`semanticSearchService.ts`	#3, #4, #5

🔐 共通仕様

認証・認可

認証方式: JWT + Session Hybrid

必須ヘッダー:

```
Authorization: Bearer maiml_session-{sessionId}_{timestamp}
```

organization_id の取得:

- JWTセッションから自動取得（リクエストに含める必要なし）
- `authMiddleware` がセッション情報を検証し `req.user.organizationId` に格納

権限要件:

- すべてのAPI: 認証ユーザーであればロール不問でアクセス可能
- マルチテナント分離により自組織のデータのみ取得・操作可能

統一レスポンス形式

成功レスポンス:

```
{
  "success": true,
  "data": { ... }
}
```

エラーレスポンス:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "エラーメッセージ"
  }
}
```

📋 API詳細仕様

1. セマンティック検索 ✅

エンドポイント: `POST /api/v1/semantic-search/search`

説明: 自然言語クエリをもとに、組織内のMAIMLファイルに含まれるトランジションセットを4軸ベクトルで類似度検索する

認証: 必須

呼び出し元: `TransitionSetSearchModal.tsx`（検索実行ボタン）、`useSemanticSearch` フック

リクエスト

ボディ（JSON）:

```
{
  "input_query": "シリコン基板、532nmレーザー、室温",
  "transition_query": "バックグラウンド補正、ピーク検出",
  "output_query": "スペクトルデータCSV、TIFFイメージ",
  "structure_query": "1入力2出力",
  "top_k": 50,
  "weights": {
    "input": 0.25,
    "transition": 0.25,
    "output": 0.25,
    "structure": 0.25
  }
}
```

フィールド	型	必須	説明
input_query	string	△	入力条件のクエリ文字列
transition_query	string	△	実験手順のクエリ文字列
output_query	string	△	出力内容のクエリ文字列
structure_query	string	△	構造のクエリ文字列
top_k	number	×	取得件数上限（デフォルト: 20、最大: 100）
weights	object	×	4軸の重み設定（デフォルト: 各0.25）
weights.input	number	×	入力条件の重み（0.0〜1.0）

weights.transition	number	×	手順の重み（0.0〜1.0）
weights.output	number	×	出力の重み（0.0〜1.0）
weights.structure	number	×	構造の重み（0.0〜1.0）

※ `input_query` 〜 `structure_query` のうち少なくとも1つは必須

レスポンス

成功時（200 OK）：


```
{
  "success": true,
  "data": {
    "hits": [
      {
        "id": "transition-set-uuid",
        "score": 0.923,
        "project_name": "ラマン分光実験",
        "file_path": "projects/raman_experiment/raman_experiment.maiml",
        "method_id": "method_001",
        "method_name": "ラマン測定",
        "transition_id": "transition_001",
        "transition_name": "スペクトル取得",
        "input_summary": "シリコン基板、532nmレーザー",
        "transition_summary": "バックグラウンド補正",
        "output_summary": "スペクトルCSV",
        "structure_summary": "1入力1出力",
        "scores": {
          "input": 0.95,
          "transition": 0.91,
          "output": 0.87,
          "structure": 0.98
        }
      }
    ],
    "total": 12,
    "elapsed_ms": 234,
    "query_info": {
      "top_k": 50,
      "weights_used": {
        "input": 0.25,
        "transition": 0.25,
        "output": 0.25,
        "structure": 0.25
      }
    }
```

```
    }
  }
```

フィールド	型	説明
hits	array	検索結果リスト
hits[].id	string	トランジションセットID
hits[].score	number	総合類似度スコア（0.0〜1.0）
hits[].project_name	string	プロジェクト名
hits[].file_path	string	MAIMLファイルの相対パス
hits[].method_id	string	メソッドID
hits[].method_name	string	メソッド名
hits[].transition_id	string	トランジションID
hits[].transition_name	string	トランジション名
hits[].scores	object	4軸ごとの個別スコア
total	number	マッチした総件数
elapsed_ms	number	検索処理時間（ミリ秒）

エラー時:

ステータス	エラーコード	説明
400	VALIDATION_ERROR	クエリが全て未指定
401	AUTH_TOKEN_INVALID	認証エラー
503	SERVICE_UNAVAILABLE	ベクトルDBへの接続エラー
500	SYSTEM_ERROR	システムエラー

---

2. クエリ分解（AI4軸分解） ✅

エンドポイント: `POST /api/v1/semantic-search/decompose`

説明: 自然言語クエリをAI（GPT）で4軸（入力条件・手順・出力・構造）に分解する。GPT未接続時はフォールバック動作（transition_queryにそのまま入れる）

認証: 必須

呼び出し元: `TransitionSetSearchModal.tsx`（`decomposeNaturalLanguageQuery` 経由）

リクエスト

ボディ（JSON）:

```
{
  "query": "532nmレーザーでラマン測定、CLS解析、スペクトルとTIFF画像を出力"
}
```

フィールド	型	必須	説明
query	string	✅	分解対象の自然言語クエリ

レスポンス

成功時（200 OK）:

```
{
  "success": true,
  "data": {
    "q_in": "532nmレーザー、シリコン基板",
    "q_tr": "ラマン測定、CLS解析、バックグラウンド補正",
    "q_out": "スペクトルCSV、TIFFイメージ",
    "q_struct": null,
    "original_query": "532nmレーザーでラマン測定、CLS解析、スペクトルとTIFF画像を出力",
    "fallback": false
  }
}
```

フィールド	型	説明
q_in	string | null	入力条件の分解結果
q_tr	string | null	手順の分解結果
q_out	string | null	出力の分解結果
q_struct	string | null	構造の分解結果
original_query	string	元のクエリ

fallback	boolean	フォールバック動作が発生したか（GPT未接続時はtrue）

フォールバック時（GPT未接続）:

```
{
  "success": true,
  "data": {
    "q_in": null,
    "q_tr": "532nmレーザーでラマン測定、CLS解析、スペクトルとTIFF画像を出力",
    "q_out": null,
    "q_struct": null,
    "original_query": "532nmレーザーでラマン測定、CLS解析、スペクトルとTIFF画像を出力",
    "fallback": true
  }
}
```

エラー時:

ステータス	エラーコード	説明
400	VALIDATION_ERROR	queryが未指定または空文字
401	AUTH_TOKEN_INVALID	認証エラー
500	SYSTEM_ERROR	システムエラー

---

3. ファイルインデックス登録 ✅

エンドポイント: `POST /api/v1/semantic-search/index`

説明: 指定した1ファイルのトランジションセットをEmbeddingしてベクトルDBに登録（UPSERT）する

認証: 必須

呼び出し元: `semanticSearchService.ts`（`indexFile`）

リクエスト

ボディ（JSON）:

```
{
  "filePath": "projects/raman_experiment/raman_experiment.maiml"
```

```
}
```

フィールド	型	必須	説明
filePath	string	✅	organization_id省略の相対ファイルパス

レスポンス

成功時（200 OK）:

```
{
  "success": true,
  "data": {
    "upsertedCount": 5,
    "filePath": "projects/raman_experiment/raman_experiment.maiml"
  }
}
```

フィールド	型	説明
upsertedCount	number	UPSERTされたトランジションセット件数
filePath	string	処理されたファイルパス

エラー時:

ステータス	エラーコード	説明
400	VALIDATION_ERROR	filePathが未指定
401	AUTH_TOKEN_INVALID	認証エラー
404	FILE_NOT_FOUND	ファイルが存在しない
500	SYSTEM_ERROR	Embedding処理エラー

---

4. インデックス統計取得 ✅

エンドポイント: `GET /api/v1/semantic-search/stats`

説明: ベクトルDBのインデックス統計情報を取得する

認証: 必須

呼び出し元: `semanticSearchService.ts`（`getIndexStats`）

リクエスト

クエリパラメータなし

レスポンス

成功時（200 OK）:

```
{
  "success": true,
  "data": {
    "totalVectors": 1234,
    "indexedFiles": 87,
    "lastUpdatedAt": "2026-03-05T14:30:00Z",
    "dimensions": 1536,
    "metric": "cosine"
  }
}
```

フィールド	型	説明
totalVectors	number	ベクトルDB内の総ベクトル数
indexedFiles	number	インデックス登録済みファイル数
lastUpdatedAt	string	最終更新日時
dimensions	number	Embeddingの次元数
metric	string	類似度計算メトリクス

エラー時:

ステータス	エラーコード	説明
401	AUTH_TOKEN_INVALID	認証エラー
503	SERVICE_UNAVAILABLE	ベクトルDBへの接続エラー
500	SYSTEM_ERROR	システムエラー

---

5. ヘルスチェック ✅

エンドポイント: `GET /api/v1/semantic-search/health`

説明: セマンティック検索サービス（ベクトルDB・Embeddingモデル）のヘルス状態を確認する

認証: 必須

呼び出し元: `semanticSearchService.ts`（`checkHealth`）

リクエスト

クエリパラメータなし

レスポンス

成功時（200 OK）:

```
{
  "success": true,
  "data": {
    "status": "healthy",
    "vectorDb": {
      "status": "connected",
      "latencyMs": 12
    },
    "embeddingModel": {
      "status": "available",
      "model": "text-embedding-ada-002",
      "provider": "ollama"
    },
    "gptModel": {
      "status": "available",
      "model": "gpt-4o-mini"
    },
    "checkedAt": "2026-03-06T06:35:00Z"
  }
}
```

フィールド	型	説明
status	string	全体ステータス（`healthy` / `degraded` / `unhealthy`）
vectorDb.status	string	ベクトルDBの接続状態
vectorDb.latencyMs	number	ベクトルDBの応答時間（ms）
embeddingModel.status	string	Embeddingモデルの状態

embeddingModel.model	string	使用モデル名
gptModel.status	string	GPTモデルの状態（decompose用）

エラー時:

ステータス	エラーコード	説明
401	AUTH_TOKEN_INVALID	認証エラー
503	SERVICE_UNAVAILABLE	サービス停止中

---

6. バッチ処理開始 ✅

エンドポイント: `POST /api/v1/semantic-search/batch`

説明: 組織内の全MAIMLファイルを対象に、トランジションセットのEmbedding生成バッチ処理を開始する

認証: 必須

呼び出し元: `TransitionSetBatchProcessingPanel.tsx`（`handleStart`）

リクエスト

ボディ（JSON）:

```
{
  "forceReprocess": false,
  "maxFiles": null
}
```

フィールド	型	必須	説明
forceReprocess	boolean	×	既にインデックス済みのファイルも強制再処理する（デフォルト: false）
maxFiles	number | null	×	処理対象の最大ファイル数（null: 無制限）

レスポンス

成功時（202 Accepted）:

```
{
  "success": true,
  "data": {
    "batch_job_id": "job-uuid-20260306-153800"
  }
}
```

フィールド	型	説明
batch_job_id	string	開始されたバッチジョブのID

エラー時:

ステータス	エラーコード	説明
400	VALIDATION_ERROR	パラメータ不正
401	AUTH_TOKEN_INVALID	認証エラー
409	CONFLICT	既にバッチ処理が実行中
500	SYSTEM_ERROR	システムエラー

ビジネスロジック

- バッチ処理はバックグラウンドで非同期実行
- `projects/` 配下の全 `.maiml` ファイルをスキャン
- `forceReprocess=false` の場合、既にインデックス済みのファイルはスキップ
- バッチジョブIDを返却後、進捗は `/batch` （GET）で確認

---

7. バッチジョブ一覧取得 ✅

エンドポイント: `GET /api/v1/semantic-search/batch`

説明: バッチジョブの一覧を取得する

認証: 必須

呼び出し元: `TransitionSetBatchProcessingPanel.tsx` (`fetchData` → `listBatchJobs`)

リクエスト

クエリパラメータなし

レスポンス

成功時（200 OK）:


```
{
  "success": true,
  "data": {
    "jobs": [
      {
        "id": "job-uuid-20260306-153800",
        "status": "processing",
        "total_files": 87,
        "processed_files": 43,
        "failed_files": 1,
        "skipped_files": 5,
        "error_message": null,
        "created_at": "2026-03-06T06:38:00Z",
        "completed_at": null
      },
      {
        "id": "job-uuid-20260305-100000",
        "status": "completed",
        "total_files": 80,
        "processed_files": 80,
        "failed_files": 0,
        "skipped_files": 3,
        "error_message": null,
        "created_at": "2026-03-05T01:00:00Z",
        "completed_at": "2026-03-05T01:12:30Z"
      }
    ]
  }
}
```


BatchJobオブジェクト:

フィールド	型	説明
id	string	バッチジョブID
status	string	ステータス（`started` / `processing` / `completed` / `failed` / `cancelled`）

total_files	number	処理対象の総ファイル数
processed_files	number	処理済みファイル数
failed_files	number	失敗ファイル数
skipped_files	number	スキップファイル数（既存インデックス）
error_message	string | null	エラーメッセージ（失敗時）
created_at	string	開始日時
completed_at	string | null	完了日時

エラー時:

ステータス	エラーコード	説明
401	AUTH_TOKEN_INVALID	認証エラー
500	SYSTEM_ERROR	システムエラー

---

8. バッチ統計取得 ✅

エンドポイント: `GET /api/v1/semantic-search/batch/stats`

説明: バッチ処理の累計統計情報を取得する

認証: 必須

呼び出し元: `TransitionSetBatchProcessingPanel.tsx`（`fetchData` → `getBatchStats`）

リクエスト

クエリパラメータなし

レスポンス

成功時（200 OK）:

```
{
  "success": true,
```

```
  "data": {
    "total_jobs": 5,
    "total_files_processed": 412,
    "total_files_failed": 3,
    "total_embeddings_generated": 1847,
    "last_batch_at": "2026-03-06T06:38:00Z"
  }
}
```

フィールド	型	説明
total_jobs	number	実行したバッチジョブの総数
total_files_processed	number	処理済みファイルの累計数
total_files_failed	number	失敗ファイルの累計数
total_embeddings_generated	number	生成したEmbeddingの累計数
last_batch_at	string | null	最後にバッチを実行した日時

エラー時:

ステータス	エラーコード	説明
401	AUTH_TOKEN_INVALID	認証エラー
500	SYSTEM_ERROR	システムエラー

---

9. バッチジョブキャンセル ✅

エンドポイント: `DELETE /api/v1/semantic-search/batch/:jobId`

説明: 実行中のバッチジョブをキャンセルする

認証: 必須

呼び出し元: `TransitionSetBatchProcessingPanel.tsx`（`handleCancel` → `cancelBatchJob`）

リクエスト

パスパラメータ:

パラメータ	型	必須	説明

jobId	string	✅	キャンセルするバッチジョブID

リクエストボディ: なし

レスポンス

成功時（200 OK）:

```
{
  "success": true,
  "data": {
    "job_id": "job-uuid-20260306-153800",
    "status": "cancelled"
  }
}
```

エラー時:

ステータス	エラーコード	説明
401	AUTH_TOKEN_INVALID	認証エラー
404	NOT_FOUND	バッチジョブが存在しない
409	CONFLICT	既に完了・失敗・キャンセル済みのジョブ
500	SYSTEM_ERROR	システムエラー

🔒 セキュリティ考慮事項

マルチテナント分離

- 検索・インデックス登録・バッチ処理はすべて自組織のデータのみ対象
- ベクトルDBの検索クエリに `organization_id` フィルタを必ず含める
- ファイルパスは `storageService` 経由でorganization_idを自動付与

ベクトルDBアクセス

- ベクトルDBへの直接アクセスは禁止（バックエンドAPI経由のみ）

- Embedding APIキーはサーバーサイドのみで管理（フロントエンドに公開しない）

📚 関連ドキュメント

- API一覧 - セクション17を参照
- セマンティック検索サービス
- TransitionSetSearchModal
- 型定義

📝 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-03-06	初版作成（ProjectManagement画面調査によりAPI仕様を確認・文書化）	Cline
		- API #1: セマンティック検索 POST /search	
		- API #2: クエリ分解 POST /decompose	
		- API #3: ファイルインデックス登録 POST /index	
		- API #4: インデックス統計取得 GET /stats	
		- API #5: ヘルスチェック GET /health	
		- API #6: バッチ処理開始 POST /batch	
		- API #7: バッチジョブ一覧取得 GET /batch	
		- API #8: バッチ統計取得 GET /batch/stats	
		- API #9: バッチジョブキャンセル DELETE /batch/:jobId	

作成者: Cline
作成日: 2026-03-06 15:38
最終更新: 2026-03-06 15:38